

Lab 1: Training a Word2Vec Model on a Small Corpus

Lab Objective

By the end of this lab, students will be able to:

1. **Understand** the fundamental concept of word embeddings and how words can be represented as numerical vectors.
 2. **Implement** a complete text preprocessing pipeline using industry-standard NLP libraries.
 3. **Train** a Word2Vec model from scratch on a custom dataset using Gensim.
 4. **Explore** and **query** the trained model to discover semantic relationships between words (similarity, analogies).
 5. **Connect** this foundational concept to how modern Large Language Models (LLMs) represent and understand language.
-

Libraries Used in This Lab

Before diving into the code, let us understand the tools we will be using.

1. Gensim (Generate Similar)

- **What it is:** A specialized Python library for unsupervised topic modeling and natural language processing.
- **Why we use it:** It contains a highly optimized, production-ready implementation of Word2Vec. It handles all the complex mathematical computations (neural network training, backpropagation, vector updates) behind a simple, easy-to-use interface. This allows us to focus on understanding the *concept* of embeddings rather than the low-level implementation details.
- **Key component we use:** `Word2Vec` class.

2. NLTK (Natural Language Toolkit)

- **What it is:** A comprehensive library providing tools for traditional NLP tasks. Think of it as a Swiss Army knife for working with human language data.
- **Why we use it:** It provides the `word_tokenize` function. Raw text is just a string of characters. Tokenization is the essential first step of splitting that string into a list of meaningful units (words, punctuation). Without this, we cannot feed individual words into our model.
- **Key component we use:** `word_tokenize` function.

3. string (Python Standard Library)

- **What it is:** A built-in Python module that provides constants and utility functions for working with strings.
 - **Why we use it:** It gives us `string.punctuation`, a pre-defined string containing all punctuation characters (`!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~`). We use this to filter out punctuation tokens from our tokenized sentences, ensuring our model learns from actual words.
 - **Key component we use:** `string.punctuation` constant.
-

Part 1: Setup and Data Preparation (15 mins)

In this section, we install the required libraries and create a small, custom dataset to work with.

1.1. Install Required Libraries

Run this command in your Jupyter notebook cell or terminal to install the necessary packages.

```
!pip install gensim nltk
```

- `!` tells Jupyter to run this as a shell command.
- `pip install` downloads and installs the packages from the Python Package Index (PyPI).

1.2. Import Libraries and Download NLTK Resources

```
import nltk
from nltk.tokenize import word_tokenize
from gensim.models import Word2Vec
import string
```

```
# Download the NLTK tokenizer data (required for word_tokenize to work)
# 'punkt' is a pre-trained tokenizer model that knows how to split text into sentences and words
nltk.download('punkt')
nltk.download('punkt_tab') # Additional data for the tokenizer
```

- `import nltk`: Imports the main NLTK library.
- `from nltk.tokenize import word_tokenize`: Imports only the specific tokenization function we need, keeping our namespace clean.
- `from gensim.models import Word2Vec`: Imports the Word2Vec class from Gensim's models module.
- `import string`: Imports the string module for punctuation handling.
- `nltk.download('punkt')`: Downloads the required tokenizer data. NLTK separates code from data; this step ensures the tokenizer model is available on your machine.

1.3. Create a Small Corpus

```
# A list of sentences about machine learning and AI
# This is our "corpus" - the collection of text we will train our model on
corpus = [
    "Jishnu is amazing",
    "Jishnu loves machine learning",
    "Deep learning is beautiful",
    "Word2Vec is a great way of creating word embeddings",
    "Gensim makes NLP easy",
    "Deep learning is a part of machine learning",
    "Natural language processing is a fascinating field",
    "Word embeddings are useful for semantic search",
]
print(f"Corpus loaded with {len(corpus)} sentences.")
```

- **Corpus:** In NLP, a corpus (plural: corpora) is a large and structured set of texts. Here, we create a tiny, hand-crafted corpus so the results are easy to interpret.

Part 2: Text Preprocessing (25 mins)

Raw text cannot be fed directly into a machine learning model. We must clean and structure it first.

2.1. Define a Preprocessing Function

```
def preprocess_sentence(sentence):  
    """  
    Cleans and tokenizes a single sentence.  
  
    Steps:  
    1. Lowercase: Convert all text to lowercase for uniformity (so 'Word' and 'word'  
        are treated the same).  
    2. Tokenize: Split the sentence string into a list of individual words.  
    3. Remove punctuation: Filter out tokens that are just punctuation marks.  
    """  
    # 1. Lowercase  
    sentence = sentence.lower()  
  
    # 2. Tokenize using NLTK  
    # word_tokenize() splits the sentence into words and punctuation  
    # Example: "Jishnu Loves AI!" -> ['Jishnu', 'Loves', 'AI', '!']  
    tokens = word_tokenize(sentence)  
  
    # 3. Remove punctuation  
    # List comprehension: keep only tokens that are NOT in string.punctuation  
    # string.punctuation contains all common punctuation characters  
    tokens_without_punct = [  
        token for token in tokens if token not in string.punctuation  
    ]  
  
    return tokens_without_punct
```

- **Why lowercase?** Without it, "Word" and "word" would be considered two completely different tokens by the model, splitting their representation and making it harder to learn meaning.

2.2. Apply Preprocessing to the Entire Corpus

```
# Process the entire corpus using a list comprehension
# This applies our preprocess_sentence function to every sentence in the corpus list
processed_corpus = [preprocess_sentence(sentence) for sentence in corpus]

# Display the processed corpus to see the result of our work
print("Processed Corpus (Tokenized Sentences):")
for i, sentence_tokens in enumerate(processed_corpus):
    print(f"Sentence {i+1}: {sentence_tokens}")
```

Expected Output:

```
Processed Corpus (Tokenized Sentences):
Sentence 1: ['jishnu', 'is', 'amazing']
Sentence 2: ['jishnu', 'loves', 'machine', 'learning']
Sentence 3: ['deep', 'learning', 'is', 'beautiful']
Sentence 4: ['word2vec', 'is', 'a', 'great', 'way', 'of', 'creating', 'word', 'embedd
ings']
Sentence 5: ['gensim', 'makes', 'nlp', 'easy']
Sentence 6: ['deep', 'learning', 'is', 'a', 'part', 'of', 'machine', 'learning']
Sentence 7: ['natural', 'language', 'processing', 'is', 'a', 'fascinating', 'field']
Sentence 8: ['word', 'embeddings', 'are', 'useful', 'for', 'semantic', 'search']
```

- **processed_corpus** is now a list of lists. Each inner list contains the cleaned tokens (words) from one sentence. This is the exact format Gensim's Word2Vec expects as input.

Part 3: Training the Word2Vec Model (20 mins)

Now we train our model. This is where the "magic" happens.

3.1. Train the Model

```
# Create and train the Word2Vec model
# We are creating an instance of the Word2Vec class with specific parameters
model = Word2Vec(
    sentences=processed_corpus, # Our preprocessed list of tokenized sentences
```

```
vector_size=50,          # Dimensionality of the word vectors (how many numbers represent each word)
window=3,                # Maximum distance between target and context words
min_count=1,             # Ignore words that appear less than this many times (1 means keep all)
sg=1,                    # Training algorithm: 1 for Skip-gram, 0 for CBOW
workers=4,               # Number of CPU threads to use for parallel training
epochs=50                # Number of passes (iterations) over the training data

ta
)

print("Model training complete!")
```

Explanation of Key Parameters:

Parameter	Value	Explanation
<code>sentences</code>	<code>processed_corpus</code>	The training data. Must be an iterable of iterables (list of token lists).
<code>vector_size</code>	<code>50</code>	The size of the dense vector to represent each word. Larger vectors can capture more information but require more data. In modern LLMs, this is often 768, 1024, or higher.
<code>window</code>	<code>3</code>	The number of words before and after the target word to consider as context. A window of 3 means look at 3 words to the left and 3 to the right.
<code>min_count</code>	<code>1</code>	Ignores all words with a total frequency lower than this. Useful for filtering out very rare typos or words that don't have enough data to learn a good representation.
<code>sg</code>	<code>1</code>	Skip-gram (sg=1): Predicts context words given a target word. Better for small datasets and rare words. CBOW (sg=0): Predicts a target word given its context. Faster and better for frequent words.
<code>workers</code>	<code>4</code>	Enables parallel processing to speed up training on multi-core CPUs.
<code>epochs</code>	<code>50</code>	The number of times the model sees the entire dataset. More epochs can lead to better learning but can also overfit.

Part 4: Exploring the Trained Model (25 mins)

Now we interact with our trained model to see what it has learned.

4.1. Get the Vector for a Specific Word

```
# Choose a word to inspect
word_of_interest = 'learning'

# Check if the word exists in the model's vocabulary
if word_of_interest in model.wv:
    # wv stands for "word vectors" - this is the object that stores all the vectors
    vector = model.wv[word_of_interest]
    print(f"\nVector representation for '{word_of_interest}':")
    print(vector)
    print(f"Vector length (dimensions): {len(vector)}")
    print(f"Vector data type: {type(vector)}")
else:
    print(f"Word '{word_of_interest}' not in vocabulary.")
```

- **model.wv** is a KeyedVectors object that contains all the word-vector mappings. You can query it like a dictionary.
- The output is an array of 50 floating-point numbers. This is the "meaning" of the word "learning" as captured by our model.

4.2. Find Similar Words

```
# Find the 5 words most similar to 'machine'
similar_words = model.wv.most_similar('machine', topn=5)
print("\nWords most similar to 'machine':")
for word, score in similar_words:
    print(f" {word}: {score:.4f}")
```

- **most_similar()** computes the cosine similarity between the vector for 'machine' and every other word vector in the vocabulary, then returns the words with the highest similarity scores.
- The score ranges from -1 to 1, with 1 meaning identical direction, 0 meaning no relationship, and -1 meaning opposite directions.

4.3. Perform Analogy Reasoning (The "King - Man + Woman = Queen" Test)

```
# Try a simple analogy from our corpus
# We want to find: "jishnu" is to "jishnu" as "learning" is to ?
# This is essentially finding a word similar to "learning" after removing "jishnu" and adding "jishnu" again
try:
    # positive: words to add to the vector
    # negative: words to subtract from the vector
    # topn: number of results to return
    result = model.wv.most_similar(positive=['jishnu', 'learning'], negative=['jishnu'], topn=1)
    print(f"\nAnalogy: 'jishnu' is to 'jishnu' as 'learning' is to... {result[0][0]} (score: {result[0][1]:.4f})")
except KeyError as e:
    print(f"\nAnalogy skipped due to missing word: {e}")
```

- This demonstrates vector arithmetic. The operation `vector('jishnu') - vector('jishnu') + vector('learning')` simplifies to just `vector('learning')`, so this specific example just finds the word most similar to "learning". A more meaningful analogy (if our corpus had it) would be `positive=['king', 'woman'], negative=['man']` to find "queen".

4.4. Save and Load the Model

```
# Save the model to disk for later use
model.save("my_first_word2vec.model")
print("\nModel saved as 'my_first_word2vec.model'.")

# To load it later in a new session:
# loaded_model = Word2Vec.load("my_first_word2vec.model")
# word_vector = loaded_model.wv['learning']
```

- **model.save()** saves both the model architecture and the trained weights. This allows you to resume training later or use the model in another application without retraining.
-

Part 5: Discussion and Reflection (10 mins)

Take a moment to reflect on what we have accomplished.

Key Takeaways:

- 1. **Words become numbers:** We successfully converted raw text into dense numerical vectors that capture semantic meaning.
- 2. **Similar words cluster:** Words that appear in similar contexts ("deep" and "machine" both appear before "learning") ended up with similar vectors.
- 3. **Relationships are encoded:** The vector space captures relationships between words. The analogy "king - man + woman = queen" works because these relationships exist as directions in the vector space.
- 4. **Limitations are clear:** With such a small corpus, the similarities are based on very limited context. A model trained on millions of documents would produce much richer and more accurate representations.

Connection to Large Language Models (LLMs):

The "Embedding" layer at the bottom of every Transformer model (like GPT, BERT, LLaMA) does **exactly the same thing** as Word2Vec, but with two key differences:

Feature	Word2Vec	LLM Embeddings
Context	Static: One vector per word regardless of context	Dynamic: The vector for a word changes based on surrounding words
Size	Small (50-300 dimensions)	Large (768-4096+ dimensions)
Training	Shallow neural network	Deep Transformer network with billions of parameters

This lab gives you the foundation to understand how LLMs begin their process: by converting tokens into vectors that capture meaning. The rest of the Transformer layers then refine these vectors through attention mechanisms to create context-aware representations.

Bonus Challenges (Optional)

For students who finish early or want to explore further:

1. **Try CBOW:** Change `sg=1` to `sg=0` and retrain. Compare the similarity results. Which works better for this small corpus?
2. **Add more data:** Add 5-10 more sentences to the corpus and retrain. How do the similarities change?
3. **Adjust vector_size:** Try `vector_size=10` and `vector_size=100`. How does the dimensionality affect the similarity scores?
4. **Visualize the vectors:** Use PCA or t-SNE to project the 50-dimensional vectors down to 2D and plot them using matplotlib. Do related words cluster together visually?